SRW3 Phase 1N — Abstract-Transition Simulation: Implementation, Verification, and the Honest Disposition

Verdict: `PARTIALLY COMPLETE — UNIVERSAL THEOREM NOT PROVED`**.** The completion gate (the actual mechanization of the universal reachable-committed-state theorem) is NOT met. No completion tag exists. What IS established — mechanized, log-backed, and audited — is the strongest simulation result of the project to date: a decidable abstract machine (AM3) whose invariant preservation is mechanized branch-completely, whose counting algebra over the concrete pools is fully mechanized, whose concrete honest-pipeline compositions are mechanized theorems, and whose simulation correspondence is mechanized for the init/stutter/refusal/reject/accept classes. Level III only; no consensus-integration claim anywhere in this report.

1. The scientific question (execution order §3)

Can an abstract-transition simulation bridge the gap between the concrete M3 machine and the universal reachable-committed-state safety theorem? The concrete machine and its state space: M3 (the frozen 1J rules + the L-FINDING-1 repair + the total-hash redirect + the `[owise]` gate refusal; `phase1m/semantics/srw3closure-j-m3.k`, byte-identical in this tree) with the nine-cell `<srw3closureJ>` configuration. The abstract machine and its state space: AM3 (`phase1n/semantics/abs-machine.k`) — `AState = absSt(cfgP, catP, cntG, cntE, nSlot, safe, diag)`, the decidable skeleton. The initial-state relation: `simJ` projects the genesis to `absGenesis`. The transition relations: M3's ten concrete classes; AM3's `absStepJ` (the exact class-mirror, `[owise]`-partitioned, total). The simulation relation: `simJ` (concrete cell projections -> AState), with the direction concrete -> abstract and the invariant-preserving abstract step as the target. Committing transitions: exactly the accept-commit class (the concrete rule's own 24-conjunct guard); refusals: the explicit diagnostic-writing classes; stuttering: the init-refuse / commit-reject classes (complete machine stutters) plus the ONE declared stall class (exec-on-unpinned — the honest mirror of the concrete machine's missing `pExecJ` rule, carried by `absExecutable`). Invariant transfer: `ABSInv`'s `safe` bit is established only by the init guard's COMPUTED `SafeJ(P,.Map)` and carried only by the accept-commit's COMPUTED `SafeJ`-on-derived-post bit — never assumed in its own induction step. Inherited assumptions: the frozen R3 configuration commitment (`CfgEqR3`/`ProtoRootR3`), Phase 1J's bounded execution fragment (`ExecFragmentJ`), and the anchor discipline — all used verbatim as the SIM claims' hypotheses. Finite-reachability induction: SUPPORTED at the abstract level per-class (branch-complete) and demonstrated for concrete pipelines; NOT supported for fully-symbolic programs (the composition boundary, §4).

2. What is formally proved (74 claims, every one log-backed)

The ABS ladder (61 claims, gate GREEN): the base (`ABS-INIT-BASE`: the genesis + the abstract init-accept lands on the initialized literal); the ten-class step partition with exact successors; the BRANCH-COMPLETE preservation family — for every command class x every constructor-bit valuation x both `ABSInv` branches, over ALL field valuations, including the complement cases (the empty-pool mint takes the refusal class; the low-pool accept takes the reject class) and the branch-exclusivity witnesses; the concrete honest-pipeline composition theorems (`ABS-RUN-HONEST-PIPELINE[-SHAPE]`: init -> exec-mint -> gate-mint -> commit-accept composed through the fold, with real multi-step unfolding); the refusal-chain composition (the machine's own diagnostic-overwrite ordering — the sweep caught the author's wrong expectation and the machine taught the correct one); and the negative controls: the unsafe-bit accept CANNOT carry the safety bookkeeping (FAILED as designed) and the gate-refusal rule CANNOT fire on a mint-satisfying state (FAILED as designed — the non-overlap control).

The SIM ladder (22 claims): the simulation shape claims for the init-accept/init-refuse/commit-accept/gate-refuse/commit-reject classes (each taking the machine's own guard verbatim — the hash-class equalities discharged as HYPOTHESES, never evaluated) and the source-dispatch class; ALL NINE counting-algebra claims (`SIM-ALG-*`: `simJ(post-shape) ==K absStructOf(absStepJ(simJ(pre-shape), cmd))` over symbolic pools — the pool arithmetic fully mechanized); and the negative controls (the mint-on-forbidden-source and commit-on-stale-slot destinations are unreachable).

The Python mirror (exhaustive): `abs_mirror.py` sweeps the entire abstract grid (384 states x 26 command bit-valuations): per-class preservation holds exhaustively; the K-agreement spots match; the domain boundary is exactly the four exec-on-unpinned classes; and the weak-machine control (the computed-safe bit replaced by the presented bit) breaks the invariant in 48 states — the guard is load-bearing, confirmed independently of K.

3. What is refuted, demonstrated, assumed, or not mechanized

Refuted (by the designed controls): the unsafe-bit accept; the refusal-on-mint; the mint-on-forbidden-source; the commit-on-stale-slot. Demonstrated (concrete execution, not proof): the M4 runtime smoke; the frozen M2/M3 LLVM T-series and the Python mirrors of 1M (retained evidence); the exhaustive mirror sweep. Assumed (declared, per-execution discharged): the R3 configuration commitment, the bounded execution fragment, the anchor, and — the material one — `SafeJ`'s agreement with the abstract `safe` bit (the conformance obligation is discharged by the concrete guard per execution; it is NOT a mechanized theorem — the 1L stuck-symbolic class, unchanged). Not mechanized: the universal theorem over fully-symbolic programs; the committed-prefix corollary; the symbolic mint/gate shape claims.

4. The exact status of the universal theorem

NOT PROVED — and for the first time, the boundary is precisely located with genuine residuals. `ABS-UNIV-STEP` (the structural `[circularity]` formulation over `absRunF`/`absExecutable`) fails with the residual `ABSInv(absRunF(RUN, absSt(AX0..AX6)))`: the abstract step function cannot dispatch on a SYMBOLIC command constructor (K function equations match by pattern), so the tail obligation cannot be discharged inside one kprove claim. The unpinned variant (`ABS-UNIV-TAIL`) establishes that kore's circularity does NOT self-apply (no vacuous instant closure — the runner's `WarnTrivialClaim` guard verifies), so the boundary is real, not a formulation artifact. The mechanized surrogate: the branch-complete preservation family (the per-class universal) + the concrete-pipeline composition theorems + the exhaustive Python sweep. The same boundary blocks `ABS-PREFIX-SAFE` (residual of record). A backend with structural induction over symbolic constructors (or a claim-circularity system provable in one claim) would close the composition; the recorded residuals state exactly what such a backend must discharge.

5. The 1M audit finding (Finding 1N-F1)

The 1M universal-side claims `MD-INV-ALLPATH-REACHABILITY` and `MD-INV-COMMITTED-PREFIX-SAFE` are recorded as `FAILED(rc=1)` genuine attempts, but their logs of record contain `Warning (WarnTrivialClaim): Claim proven without rewriting` + `#Top` — both claims closed VACUOUSLY (destination = source; the safety conclusion already inside the hypothesis). The universal composition was never genuinely attempted at the kore level in 1M. The 1M disposition (NOT MECHANIZED) stands a fortiori; the diagnosis changes; 1N's runner enforces the guard so this cannot recur. (audit/1m-misclassification-finding.md)

6. The blockers, classified (execution order §4)

- B5 (condition-equivalence over stuck-defined hash terms): STRUCTURALLY
- B5' (the symbolic search cost of the mint/commit cascade): LIVE —
- B2 (the destination-constraint idiom): BYPASSED for the accept class —
- The circularity/vacuity class: contained — `SafeJ` never assumed in its
- NEW (the 1N composition boundary): the symbolic-command dispatch —

7. Reproduction

```
bash scripts/reinstall_toolchain_1m_r3.sh # the pinned toolchain
bash phase1n/proofs/run_n_abs_proofs.sh <out> "" /tmp/n-abs # 61 claims
bash phase1n/proofs/run_n_sim_proofs.sh <out> "" /tmp/n-abs # 22 claims
python3 phase1n/python/abs_mirror.py # the sweep
python3 phase1n/scripts/summarize_1n.py # fail-closed
```

Every claim's raw log: `phase1n/transcripts/k/kprove_<CLAIM>.log`
(timeouts of record are 0-size logs listed in
`transcripts/k/README-0-size-logs.txt`, per the 1M runner-discipline
convention). The union inventories: `claims-status.csv`,
`claim-classification.csv`.

8. The completion gate, applied

The universal reachable-committed-state theorem is NOT mechanized (§4); the phase is therefore `PARTIALLY COMPLETE — UNIVERSAL THEOREM NOT PROVED` under its own gate; no `phase1n-complete` tag exists; the strongest proved results (74 claims) are preserved with their logs and never relabeled. No end-to-end Ethereum security claim is made or implied: the results concern the explicitly modeled execution fragment under the declared R3/1J assumptions, at evidence Level III.